Intermediate (build the loop yourself) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — Doc06's round trip, repeated
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — Doc06's round trip, repeated

Basic Version

Doc06's Failure exercise did one round trip: ask the model, run the tool it asked for, send the result back with a `{"role": "tool", ...}` message, ask again. An agent is that same round trip **in a loop** — it keeps going until the model answers without asking for a tool.

Copy Doc06's Build Task files into `practice/`, unchanged: `tools.py`, `http_client.py`, `exceptions.py`, `logging_setup.py`. Run everything from inside `practice/`.

- `model.bind_tools([add])` — the model can now ask for `add`.
- `messages` — the growing list: the task, each model response, each tool result.
- `for step in range(5):` — a loop that can never run forever.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Each turn of the loop has three parts — the names from the ReAct topic:

- **Think:** `response = model_with_tools.invoke(messages)`, then `messages.append(response)`.
- **Act:** if `response.tool_calls` is empty, the model is done — return `response.content`. Otherwise run every call with `tool.invoke(call["args"])`.
- **Observe:** append `{"role": "tool", "content": result, "tool_call_id": call["id"]}` for each call.

Keep a `steps` list (step number, tool, result), return it in an `AgentResult` dataclass, and when the loop runs out, raise `MaxIterationsExceeded(message, steps)` — a named error that carries the partial log, like Doc02's `PermanentHTTPError` carries its status code.

Difference between Basic and Intermediate: Basic names the pieces. Intermediate splits each turn into think / act / observe, and adds the result, the log and the named error.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
model_with_tools = model.bind_tools([add])
messages = [the task]
for step in range(5):
    response = ask the model
    add response to messages
    if no tool calls: print the answer, stop
    for each call: run add, print it, add a tool message
```

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
run_agent(task, max_iterations) -> AgentResult:
    for step in range(max_iterations):
        think; if done: return AgentResult(answer, steps)
        for each call:
            result, error = run_tool(call, ALL_TOOLS)
            observation = result, or "Error: " + error
            log it; add a tool message; add a step dict
    raise MaxIterationsExceeded(message, steps)

run_tool(call, tools) -> (result, error):
    find_tool (Doc06) -> None? return (None, "unknown tool")
    try: return (str(tool.invoke(args)), None)
    except Exception as e: return (None, str(e))
```

Write Approach 1 with `add` only, then Approach 2 with every tool in `ALL_TOOLS`. Then three more: a factory `make_flaky_add(state)` that returns a new `@tool` failing on its first call; `run_tool_checked()` that builds the tool's Pydantic model from the arguments and calls `.invoke(checked.model_dump())`; and the same loop on the raw SDK (`client.chat.completions.create(..., tools=[schema])`, `json.loads(call.function.arguments)`). Compare against the Solution.

Difference between Basic and Intermediate: Basic is one loop for one tool. Intermediate is the reusable `run_agent()` the next exercises import.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution